

ECE2810J — Data Structures and Algorithms

Programming Assignment 2

Instructor: [Weikang Qian](#)

— SJTU GC (Summer 2026)

Notes

- Due Date: TBD
- Submission: on JOJ3 (<https://focs.gc.sjtu.edu.cn/git/ece281s2>)

1 Introduction

In this project, you will be asked to implement a modern, high-performance, STL-like hash table based on the idea of *Swiss Table*.

Hash tables are one of the fundamental data structures in software engineering. They are used almost everywhere. A high-performance hash table can save tech companies plenty of computational resources. In the Standard Template Library, hash tables are implemented as `std::unordered_map` [1], which was introduced in the C++11 standard. It is an associative container that contains key-value pairs with unique keys. Search, insertion, and removal of elements have average $\mathcal{O}(1)$ time complexity. It is named “unordered” because there already exists an ordered data structure of key-value pairs based on the red-black tree, `std::map`. Internally, separate chaining is used as the hash collision resolution strategy. If you are interested, you may refer to LLVM’s libc++ source code at https://github.com/llvm/llvm-project/tree/main/libcxx/include/__hash_table.

In 2017, engineers at Google proposed a new hash table design: *Swiss Table* [3]. Swiss Table is based on open addressing, which is more cache-friendly than separate chaining. In 2018, the C++ implementation of Swiss Table was open-sourced in the Abseil library [4]. Later, languages including Golang and Rust have adopted the algorithm in their standard libraries. In 2019, Rust replaced the implementation of `HashMap` with `hashbrown`, a port of Swiss Table in Rust, in version [Rust-1.36.0](#). In 2022, the ByteDance Programming Language team opened the issue [runtime: use SwissTable](#) in the official repo. Benchmarks show that by switching to Swiss Table, Go achieved a huge performance improvement of its `map` [5].

2 Programming Assignment

2.1 A Minimal Introduction to Swiss Table

The following blog provides an overview of how Swiss Table works and why it is efficient: <https://go.dev/blog/swisstable>. You are encouraged to read it before writing code.

2.1.1 Concept: Hash Function

In this project, a hash function takes an input of some type and produces a `std::size_t` value as the output. It corresponds to the **hash code** we mentioned in the lecture.¹ On most 64-bit platforms, `std::size_t` is 64 bits.

In C++, you can hash values of many standard types with `std::hash<T>`. Try the following code yourself and see what you get.

¹If you are taking or have taken ECE4750J, note that the hash function here is different from a cryptographic hash function.

```

1 #include <stdio>
2 #include <string>
3 std::printf("%zu\n", std::hash<int>{}(123)); // %zu: print as unsigned decimal
4 std::printf("%zu\n", std::hash<std::uint64_t>{}(123ull));
5 std::printf("%zx\n", std::hash<std::string>{}("123")); // %zx: print as hexadecimal

```

2.1.2 Overview

To give you a general feeling about what we are doing, Table 1 shows the final layout of a Swiss Table. Note that for a hash table, it generally stores key-value pairs, but for simplicity, we do not show the values in the following tables.

Table 1: An example layout of a Swiss Table.

	Group 0						Group 1		
Slot	0	1	2	3	4	...	0	1	...
Key	"saki"	"miku"	"kaguya"	<deleted>	<empty>		"akane"		
Ctrl	0x08	0x03	0x03	0x80	0xfe		0x7f		

2.1.3 Group and Slot

If you have taken ECE3700J or other hardware-related courses, you may know about *associative caches*. Swiss Table adopts a similar strategy. It consists of a number of groups, and each group contains a fixed number of slots (usually 8 or 16). The 64-bit hash of an item is split into two parts. The first part, h_1 (57 bits), is used to address a *group*. The second part, h_2 (7 bits), is used for a faster check of whether an item may exist in a group. You can either use the top 7 bits or the lowest 7 bits as h_2 , and use the remaining bits as h_1 .

For example, suppose the string "saki" has the hash 0x12340008. Then, we use the lowest 7 bits as h_2 (0x08), and the remaining bits as h_1 (0x246800 after shifting right by 7 bits). Suppose we have 8 groups. The element "saki" will then be placed into group "0x246800 % 8", which is group 0. Now suppose we have the following elements:

Table 2: Example calculation of h_1 , h_2 , and group index in Swiss Table.

Item	Hash Value	h_2 (lowest 7 bits)	h_1 (hash $\gg 7$)	Group Index ($h_1 \bmod 8$)
"saki"	0x12340008	0x08	0x246800	0
"miku"	0x43210003	0x03	0x864200	0
"akane"	0x132400ff	0x7f	0x264801	1
"kaguya"	0x34120003	0x03	0x682400	0

The layout will look like

Table 3: Example layout of groups.

	Group 0				Group 1		
Slot	0	1	2	...	0	1	...
Key	"saki"	"miku"	"kaguya"		"akane"		
h_2	0x08	0x03	0x03		0x7f		

Note that a new item can be placed in **any** slot as long as the slot is not occupied. In particular, h_2 does **not** decide which slot the item goes into; it is only used for speeding up the search (see Section 2.1.7 for details).

2.1.4 Control Byte and Control Word

Each slot can be in one of three states: empty, occupied, or deleted.

In addition to the 7 bits used for h_2 , we need one extra bit to indicate if the slot is occupied or not. The 8 bits form a *control byte* of a slot. Suppose that the highest bit of a control byte is set to 0 for an occupied slot and 1 for empty and deleted slots. If the highest bit is 0, we store h_2 in the remaining 7 bits of the control byte. If the highest bit is 1, we distinguish the empty and deleted slots by the remaining 7 bits. As an example, we use **0xfe** (**0b11111110**) to represent an empty slot and **0x80** (**0b10000000**) to represent a deleted slot. Table 4 shows an example of control bytes in a Swiss Table. Suppose that each group has 8 slots. The 8 control bytes of the slots form a 64-bit *control word* of the group.

Table 4: Example calculation of control bytes (Ctrl) in Swiss Table.

Item	Hash Value	h_2	Ctrl Byte
"saki"	0x12340008	0x08 / 0b0001000	0b00001000
"miku"	0x43210003	0x03 / 0b0000011	0b00000011
"akane"	0x132400ff	0x7f / 0b1111111	0b01111111
"kaguya"	0x34120003	0x03 / 0b0000011	0b00000011
<deleted>			0b10000000
<empty>			0b11111110

2.1.5 Operations

Let k be the key we want to search, insert, or delete. A group “contains $h_2(k)$ ” if at least one occupied slot in the group has a control byte whose lower 7 bits equal $h_2(k)$.

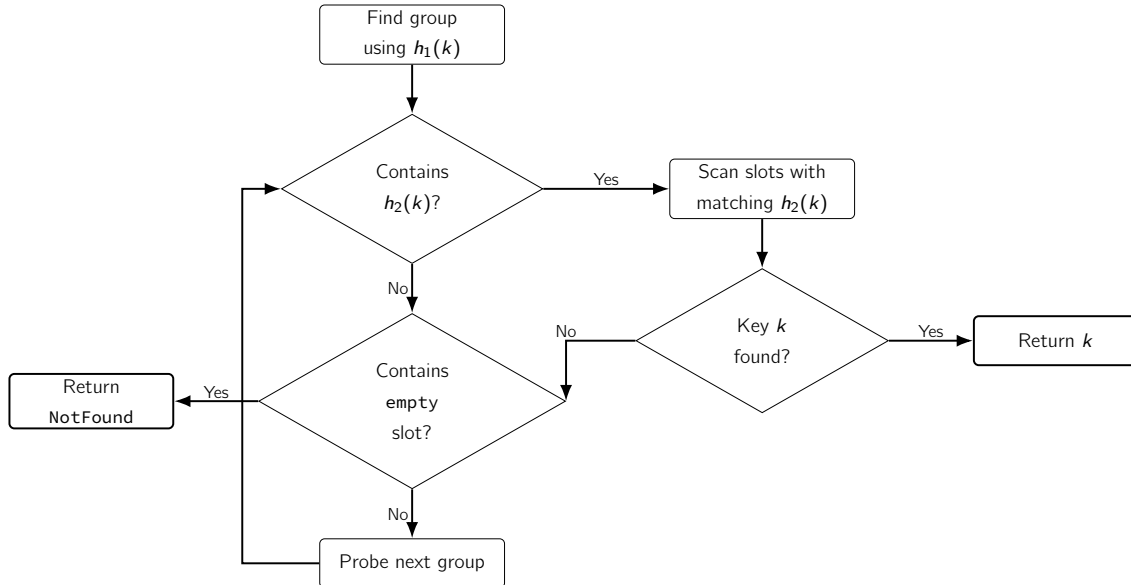


Figure 1: Search procedure.

Search: Fig. 1 shows the search procedure. It first finds the group using $h_1(k)$. If the group contains $h_2(k)$, we scan the slots whose control bytes match $h_2(k)$ to determine whether one of these slots contains the key k , since a key

different than k may share the same h_2 as k . If k is found in this group, return it. Otherwise (i.e., the group does not contain $h_2(k)$, or the group contains $h_2(k)$ but not the key k):

- if the group contains an **empty** slot, return **NotFound**;
- otherwise, probe and check the next group against $h_2(k)$.

Insert: Search for the key k by following the same search procedure mentioned above with a slight change described below. If the key already exists, you do nothing (i.e., the old value should not be replaced). Otherwise, you will perform insertion. Specifically, during the search process, record the first **deleted** or **empty** slot encountered, and insert the new element into that slot.

Delete: Search for the key k by following the same search procedure mentioned above with a slight change described below. If the key k is not found, you do nothing. Otherwise, you find the key at a certain group. If the group contains any **empty** slot, mark the slot containing k as **empty**. Otherwise, mark the slot as **deleted**. Try figuring out why this is the correct way to do.

Rehash: collect all elements and reinsert them into a (usually larger) table. When $\frac{\text{size}}{\text{\#slots}}$ (the number of elements divided by the number of slots) has exceeded a certain value (the *load factor*), your table should be rehashed.

2.1.6 Probing

The standard implementation of Swiss Table uses a special version of quadratic probing. Let $g_0(k) = h_1(k) \bmod M$ be the starting group, where M is the number of groups. The i -th group in the probing sequence is

$$g_i(k) = \left(g_0(k) + \frac{i^2 + i}{2} \right) \bmod M.$$

We require M to be a power of 2. Think about the following questions yourself:

- Why not use linear probing?
- Suppose $g_0(k) = 0$ and $M = 8$, write down $g_1(k), g_2(k), \dots, g_7(k)$. Have you noticed anything special?
- What is special about this probing function when M is a power of 2?

2.1.7 Optimization

In more advanced implementations, SIMD (Single Instruction, Multiple Data) operations are used to accelerate the lookup process. This is not required in our course. Instead, you are encouraged to use bit operations to achieve high performance, which is known as SWAR (SIMD Within A Register). In addition, advanced features such as incremental rehashing are NOT required.

You may wonder why we use exactly 7 bits for h_2 . This choice is related to SIMD/SWAR optimization. Recall that each slot has one control byte, and each group has one control word of size 64 bits (if group size is 8). This makes it convenient to use a single `uint64_t` to store the control word. We can then use bit operations to accelerate various operations. Same as the example above, we use `0xfe` (`0b11111110`) to represent an empty slot and `0x80` (`0b10000000`) to represent a deleted slot in this example.² If we want to check whether a group is full, we can compute the bitwise AND between the control word and `0x8080808080808080`. If the result is zero, then the group is full. Try exploring how to use bit operations to match other patterns yourself!

²You may choose a different representation if you prefer.

Table 5: Control byte (Ctrl) of slots.

	Group 0						Group 1		
Slot	0	1	2	3	4	...	0	1	...
Key	"saki"	"miku"	"kaguya"	<deleted>	<empty>		"akane"		
Ctrl	0x08	0x03	0x03	0x80	0xfe		0x7f		
& 0x80	0x00	0x00	0x00	0x80	0x80		0x00		

2.2 The Template

2.2.1 Overview

In the project starter code `swiss.hpp`, we define a template class for you:

```
1  template <class Key, class T, class Hash = std::hash<Key>,
2      class KeyEqual = std::equal_to<Key>>
3  class SwissTable
```

Key and T are the types of the key and value, respectively, stored in the hash table. Hash is the type of a function object, whose instance returns the hash value of a key. The standard library defines a class template `std::hash<T>`, which can be used as the default. Similarly, `KeyEqual` is another type of a function object, whose instance returns whether two Key objects are equal.

We will use the default hash and comparator template parameters provided by you (i.e. `std::hash<Key>` and `std::equal_to<Key>` by default) in ALL cases. You can choose either to keep the default, or wrap `std::hash<Key>` and add some other operations. We WILL construct test cases with certain patterns that could lead some naive implementation to TLE (Time Limit Exceeded). If you want to define your own hash function and key-equal function, you can refer to the following definitions:

```
1  template<class Key>
2  struct hash<Key> {
3      size_t operator()(const Key &key) const;
4  }
5
6  template<class Key>
7  struct equal_to<Key> {
8      bool operator()(const Key &lhs, const Key &rhs) const;
9  }
```

2.2.2 Iterators

You need to implement STL-style iterators for your Swiss Table. The boilerplate code is provided to you.

```
1  class Iterator {
2  public:
3      // define the category of the iterator. you can ignore this line
4      using iterator_category = std::forward_iterator_tag;
5  }
```

```

6 private:
7     friend class SwissTable;
8     SwissTable* table_ = nullptr;
9 };

```

After implementing the iterators, the hash table will support basic iteration and range-for loops[2]. An iterator should always point directly to an occupied slot, except for the end iterator; when incremented, it should skip empty and deleted slots. In the traditional iterator style, you can iterate over the hash table by:

```

1 for (auto it = table.begin(); it != table.end(); ++it) {
2     std::cout << it->first << " " << it->second << std::endl;
3 }

```

In C++11, you could use the alternative:

```

1 for (auto &item : table) {
2     std::cout << item.first << " " << item.second << std::endl;
3 }

```

With C++17, you can even do it in a more graceful way with the *structured binding* feature:

```

1 for (auto &[key, value] : table) {
2     std::cout << key << " " << value << std::endl;
3 }

```

The order of the output of the above code is arbitrary and is dependent on implementation. We will not test the internal order of the key-value pairs in your hash table (returned by iteration), since they're meant to be "unordered". To make your life easier, **it is fine for your iterators to become invalid after a rehash**.

2.3 (Optional) High Performance of Swiss Table

This part is optional. If you are interested in CPU optimization and the design of the algorithm, you are recommended to read and try everything yourself.

We have mentioned that Swiss Table is more cache-friendly than separate chaining approach, which is an intuitive guess. But is it the real reason behind the high performance of Swiss Table? After you have passed all the test cases, you can try to profile your implementation using `perf`(Linux) or `xctrace`(MacOS). If you are new to profilers, feel free to let generative AIs process and explain your report!

Try answering:

- Why is Swiss Table much faster?
- What are the disadvantages of Swiss Table compared to separate chaining based approaches?

3 Implementation Requirements and Restrictions

This project makes heavy use of features from modern C++ (templates, move semantics, namespaces, iterators, etc.). If you are unfamiliar with these features, you are recommended to attend the recitation class. TAs will introduce features involved in this project and why they are designed in this way. You are also encouraged to rely on AI tools and `cpp-reference` to learn those features yourself.

3.1 Requirements

There are NO constraints on how you implement the Swiss Table. You can

- Choose an arbitrary probing strategy (standard: quadratic probing described in Section 2.1.6)
- Choose an arbitrary group size (standard: 8 in Golang, 16 in Abseil)
- Choose an arbitrary load factor (standard: 7/8)

You are encouraged to use the setups from mature libraries (e.g., Go `map` and Abseil C++ `flat_hash_map`) since those setups are well-benchmarked and have proven to be efficient.

- You must make sure that your code compiles successfully on a machine with Linux operating system with `clang++` and the options `-std=c++20 -Wconversion -Wall -Werror -Wextra -pedantic`.
- You should not change the definitions of the functions and variables, as well as the `public` or `protected` keywords for them in `swiss.hpp`. However, you ARE allowed to add or remove qualifiers that have no effect on the behavior of the interfaces, including `constexpr`, `noexcept`, `inline`, and attributes like `[[nodiscard]]`. You are NOT allowed to remove `const`.
- You can define helper functions and new variables. Don't forget to mark them as `private` or `protected`.
- You should only edit the file `swiss.hpp`.
- You can use any header file defined in the C++20 standard. You can use `cppreference` as a reference.
- You should NOT use any map-like STL containers.

You only need to implement the methods (functions) marked with "TODO" in the file `swiss.hpp`. Here is a list of the methods (functions):

- Basic iterator operations
- Constructor, destructor, and assignment operator
- `empty`, `size`
- `find`, `at`, `contains`, `count`. They are all for searching.
- `insert`
- `erase`
- `operator[]`
- `reserve`

All interfaces are designed to be consistent with `std::unordered_map`. Please refer to the descriptions of these functions in the `swiss.hpp`. In case you are not familiar with `std::unordered_map`, a detailed description of all the methods above can be found in the appendix.

3.2 Memory Leak

Your program should NOT have memory leaks. You may want to use valgrind to check for memory leaks. You need to install valgrind first if your system does not have it installed. The command to check memory leak is:

```
valgrind --leak-check=full <COMMAND>
```

You should replace <COMMAND> with the actual command you use to issue the program under testing. For example, if you want to check whether running program

```
./main < input.txt
```

causes memory leak, then <COMMAND> should be `./main < input.txt`. Thus, the command will be

```
valgrind --leak-check=full ./main < input.txt
```

Your program is expected to have no memory issues or undefined behaviors. You may use valgrind or sanitizers to check on your own. The compiler arguments are `-fsanitize=address,undefined`. Note that when benchmarking, you should turn off sanitizers.

3.3 Test Cases

The test cases are split into two parts:

- 30 cases will be used to test your correctness. For these cases, we will apply a loose time/memory bound so that as long as you have a correct implementation, you would pass the tests. Your code will be compiled by

```
1 clang++ -std=c++20 -O0 -Wconversion -Wall -Werror -Wextra -pedantic
   ↪ -fsanitize=address,undefined -o main main.cpp
```

- 12 cases will be used to test your performance. For these cases, we will apply a tight time/memory bound so that you need to optimize your implementation properly to pass the tests. The bound is set somewhere between the TAs' implementation and `std::unordered_map`. Your code will be compiled by

```
1 clang++ -std=c++20 -O2 -Wconversion -Wall -Werror -Wextra -pedantic -o main main.cpp
```

To make it clearer how fast your implementation should be, we have run a benchmark locally (MacBook Air with Apple M2, using a power adapter) for your reference. The results are shown in Table 6. Note that the benchmark was done using some old test cases. We put it here just to give you a rough feeling. It is possible that you cannot achieve as large an improvement on some cases as shown here due to changes in the test cases.

Hints:

- Case 31,32,33: random/sequential insertions and random reads
- Case 34,35: sequential insertions, random erasures/assignments and random reads
- Case 36,37,38: mixed operations
- Case 40,41: iteration over sparse map/dense map

You are recommended to run your code more than once on JOJ to reduce randomness.

4 Submission and Grading

You only need to submit the file `swiss.hpp` on JOJ3. Name it as `/p2/swiss.hpp` in your repo. Use `p2` as the scope to test your implementation on push.

Your program will be graded along several criteria:

Table 6: Benchmark: Swiss vs unordered_map.

Case	TAs' Impl (ms)	std (ms)	Improvement
30	30.380	40.356	+24.7%
31	74.164	474.332	+84.4%
32	116.753	143.151	+18.4%
33	68.747	334.760	+79.5%
34	114.273	177.866	+35.8%
35	222.255	289.704	+23.3%
36	307.860	441.162	+30.2%
37	248.325	772.580	+67.9%
38	70.944	240.336	+70.5%
39	3.800	5.143	+26.1%
40	168.466	218.389	+22.9%
41	111.322	317.749	+65.0%

4.1 Functional Correctness

Functional correctness is determined by running a variety of test cases against your program, and checking your solution using our automatic testing program. In addition, some behaviors cannot be directly tested using JOJ because they depend on your implementation. For these behaviors, including the iterators returned by insertion and erase operations, we will manually check your code to ensure their correctness.

4.2 Implementation Constraints

We will grade Implementation Constraints to see if you have met all of the implementation requirements and restrictions. In this project, we will also check whether your program has memory leak/memory errors/undefined behaviors. For those programs that behave correctly but have those errors, we will deduct some points.

4.3 General Style

General Style refers to the ease with which TAs can read and understand your program, and the cleanliness and elegance of your code.

4.4 Performance

We will test your program with some large test cases. If your program is unable to finish within a reasonable amount of time, you will lose the performance score for those test cases.

5 Acknowledgements

Project designed and authored by [Qizhen Sun](#) for ECE2810J Su26, with introductory material adapted from previous versions of ECE2810J P2 by Yihao Liu, an alumnus of JI and the chief architect of JOJ, reviewed by Tianle Xue and Prof. Weikang Qian.

References

[1] std::unordered_map - cppreference : https://en.cppreference.com/w/cpp/container/unordered_map

- [2] Range-based for loop - cppreference: <https://en.cppreference.com/w/cpp/language/range-for>
- [3] CppCon 2017: Matt Kulukundis “Designing a Fast, Efficient, Cache-friendly Hash Table, Step by Step: <https://www.youtube.com/watch?v=ncHmEUmJZf4>.
- [4] Abseil Common Libraries (C++): https://github.com/abseil/abseil-cpp/blob/master/absl/container/internal/raw_hash_set.cc
- [5] Microbenchmarks over map - godev: <https://go.dev/issue/54766#issuecomment-2542444404>

Appendix: Methods to Implement

Note that some methods are denoted as optional. If you are interested in, you are encouraged to implement them, but they will not be tested.

Iterator: an iterator ALWAYS points to a valid item directly (except the end iterator).

```
1  class Iterator {
2  public:
3      // add your constructor
4      Iterator() = default;
5
6      // return the reference to the element the iterator points to
7      kvpair_t& operator*() const noexcept {}
8
9      // return the pointer to the element the iterator points to
10     kvpair_t* operator->() const noexcept {}
11
12     // ++it. increment the iterator and return the iterator AFTER incrementing
13     constexpr Iterator &operator++() noexcept {}
14
15     // it++. increment the iterator and return the iterator BEFORE incrementing
16     constexpr Iterator operator++(int) noexcept {}
17
18     // return true if both iterators point to the same element in the same table
19     bool operator==(const Iterator& o) const noexcept {}
20 };
21 class SwissTable {
22 public:
23     // iterator interfaces
24     // return begin iterator
25     Iterator begin() noexcept {}
26
27
28     // return end iterator (the iterator AFTER the last element)
29     Iterator end() noexcept {}
30 }
```

Move Operations: Move semantics is an important feature added in C++11. It enables you to avoid the high cost of copying the whole container. Here is a short introduction to move semantics in C++:

In C++11 and later, `std::move` can be used to convert an lvalue into an rvalue reference (T&&). Rvalues include literals, temporarily constructed objects, and objects that are about to be destroyed. When an object is treated as an rvalue, the move constructor or move assignment operator may transfer its internal resources instead of copying them. For example, `std::vector` internally stores a pointer together with some metadata such as size and capacity.

```
1  std::vector<int> v1;
2  // insert values into v1
3
4  std::vector<int> v2 = v1;
5  // allocate new memory and copy all elements one by one
6
7  std::vector<int> v3 = std::move(v1);
8  // transfer the internal storage of v1 to v3
```

Conceptually, the above code is similar to the following simplified example:

```
1  struct Vec {
2      int* ptr;
3      size_t size;
4  };
5
6  // v2 = v1
7  for (size_t i = 0; i < v1.size; i++) {
8      v2[i] = v1[i];
9  }
10
11 // v3 = std::move(v1);
12 delete[] v3.ptr;
13 v3.ptr = v1.ptr;
14 v3.size = v1.size;
15 v1.ptr = nullptr;
16 v1.size = 0;
```

After a move operation, the moved-from object is still valid, but its value is unspecified. For example, after `std::move(v1)`, the vector `v1` can still be destroyed or assigned a new value, but its previous contents should not be relied upon.

Constructors, destructors, and assignment operators.

```
1  class SwissTable {
2  public:
3      // destructor: free the space you have allocated
4      ~SwissTable() {}
5
6      // clear the table but do NOT free the space
7      void clear() noexcept {}
```

```

8
9     SwissTable() = default;
10
11     // construct an empty table and reserve enough space for roughly `count` elements
12     explicit SwissTable(size_t count, const Hash &hash = Hash(),
13                          const KeyEqual &equal = KeyEqual()) {}
14
15     // constructor. insert a range of elements conveniently.
16     // if bucket_count is non-zero, use it to reserve space before insertion
17     template <std::input_iterator InputIt>
18     SwissTable(InputIt first, InputIt last, size_t bucket_count = 0,
19               const Hash &hash = Hash(), const KeyEqual &equal = KeyEqual()) {
20         if (bucket_count != 0) reserve(bucket_count);
21         for (; first != last; ++first) insert(*first);
22     }
23
24     // copy constructor
25     SwissTable(const SwissTable& other) {}
26
27     // move constructor
28     SwissTable(SwissTable &&other) {}
29
30     // copy assignment
31     SwissTable& operator=(const SwissTable& other) {
32         // TODO:
33         return *this;
34     }
35
36     // move assignment
37     SwissTable& operator=(SwissTable&& other) {
38         // TODO:
39         return *this;
40     }
41
42     // optional: swap with another table.
43     // conceptually equivalent to std::swap(*this, other)
44     void swap(SwissTable& other) {}
45 };

```

Size related:

```

1     // return true if the table is empty, false otherwise
2     [[nodiscard]] bool empty() const noexcept {}
3
4     // return the number of elements

```

```

5      [[nodiscard]] size_t size() const noexcept {}
6
7      // optional: return the capacity of the table, i.e. the maximum number of elements
8      // the table can hold without any rehash.
9      [[nodiscard]] size_t max_size() const noexcept {}

```

Lookup related:

```

1      // get a value by the key.
2      // if the key does not exist, throw std::out_of_range
3      T &at(const Key &key) {}
4
5      // const version of the one above
6      const T &at(const Key &key) const {}
7
8      // if the key exists, return the iterator associated with the key.
9      // otherwise, return end()
10     Iterator find(const Key &key) {}
11
12     // return true if the key exists, false otherwise
13     [[nodiscard]] bool contains(const Key &key) const {}
14
15     // return 1 if the key exists, 0 otherwise
16     [[nodiscard]] size_t count(const Key &key) const {}
17
18     // get a value by the key.
19     // if the key does not exist, insert {key, T{}} and return
20     // a reference to the newly inserted mapped value.
21     // this is the magic behind d[k] = v for a non-existing k.
22     T &operator[](const Key &key) {}

```

Insert:

```

1      // insert a value by rvalue reference
2      // return a pair of an iterator and a bool
3      // - if the key exists, return {iter, false} where `iter` is an
4      //   iterator pointing to the existing key.
5      // - otherwise, insert the value, return {iter, true} where
6      //   `iter` is an iterator pointing to the inserted key
7      std::pair<Iterator, bool> insert(kvpair_t &&value) {}
8
9      // insert a value by reference
10     // same requirement as above
11     std::pair<Iterator, bool> insert(const kvpair_t &value) {}
12
13     // insert many values from the iterators of other containers

```

```

14     template <std::input_iterator InputIt>
15     void insert(InputIt first, InputIt last) {}
16
17     // KEEP THIS: this method is called in the test script, do not delete it!
18     // a default implementation is provided
19     // insert a value by the key AND arguments passed to its constructor
20     template <class K, class... Args>
21     std::pair<Iterator, bool> try_emplace(K&& key, Args &&...args) {
22         // this function can be implemented differently
23         // you are encouraged to explore a better implementation that
24         // has more cpp flavor
25         return insert(kvpair_t(std::forward<K>(key), T(std::forward<Args>(args)...)));
26     }

```

Delete:

```

1     // erase a value by its iterator
2     // return the iterator AFTER the erased key
3     Iterator erase(Iterator posIt) {}

```

Reserve:

```

1     // reserve enough space for roughly `count` elements according to
2     // your chosen load factor. The exact number of slots can be larger.
3     // Leaving this empty is allowed by the interface, but your table will
4     // be slow if many elements are inserted without reserving space.
5     void reserve(size_t count) {}

```